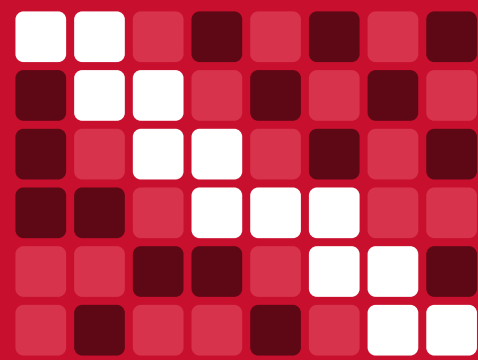


Intelligent Navigation Agent



BFS, DFS, Greedy and A* on a grid, with GUI, comparison and PEAS

How every function and algorithm in the program works, step by step

4 search algorithms	15 x 20 grid of cells	25% random walls	4 moves: up, down, left, right
------------------------	--------------------------	---------------------	-----------------------------------

What the program does

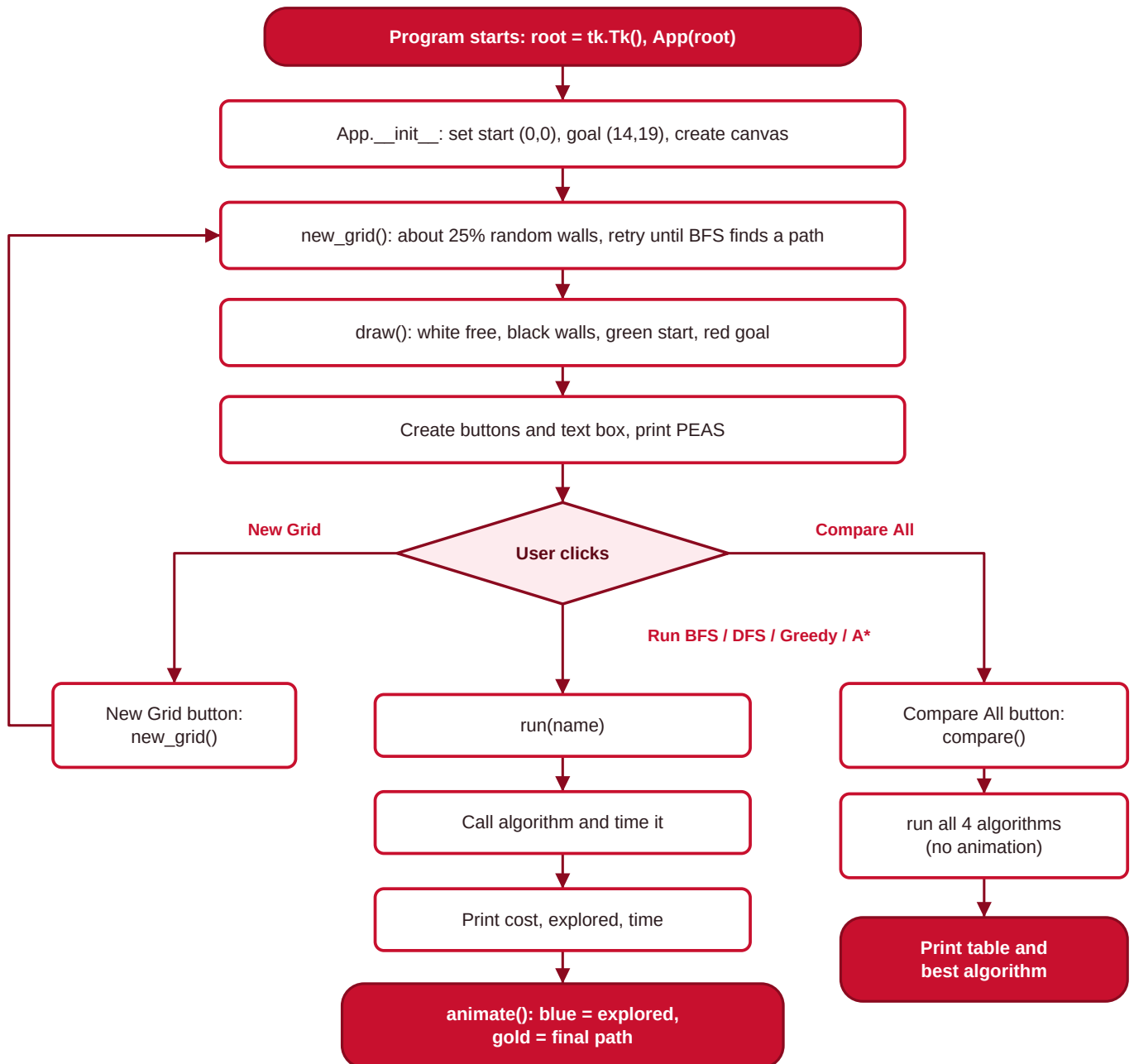
The agent starts at the top-left cell **(0,0)** and must reach the bottom-right cell **(14,19)** of a grid where about 25% of the cells are random walls. Four search algorithms solve the **same grid**, and the program reports the **path cost**, the **nodes explored** and the **time taken** for each, so they can be compared fairly.

Section	Contents
1 Big picture	Complete flow of the program from start to every button click
2 Shared helper functions	h , nbrs and trace
3 The four algorithms	BFS, DFS, Greedy Best-First, A* with a flowchart for each, and a comparison table
4 GUI and control functions	new_grid , draw , paint , run , animate , compare
5 Colour legend	What every colour on the grid means
6 PEAS of the agent	Performance, Environment, Actuators, Sensors
7 Expected result	Which algorithm wins and why

How to read the flowcharts: a **red rounded box** is a start or a return, a **white box** is a normal step, and a **pink diamond** is a yes / no decision. Arrows show the order in which the code runs.

1. Big picture

Everything starts when Python creates the Tkinter window and builds the **App** object. The program then waits for a button click. Each button leads to its own path through the code.

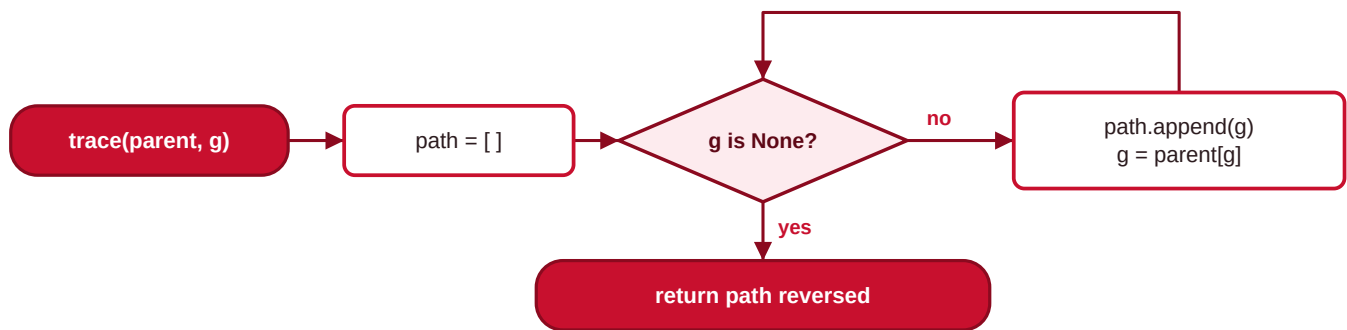


2. Shared helper functions

All four algorithms use these three small functions.

Function	What it does	Why it matters
h(a, b)	Manhattan distance: $ r1 - r2 + c1 - c2 $	The heuristic used by Greedy and A*. It never overestimates on a 4-direction grid, so A* stays optimal.
nbrs(grid, p)	Yields the up, down, left and right cells that are inside the grid and are not walls.	This is the agent's set of actions. Every algorithm calls it to expand a node.
trace(parent, g)	Walks back from the goal to the start using the parent dictionary, then reverses the list.	Turns parent links into the final start-to-goal path.

trace(parent, g) flow



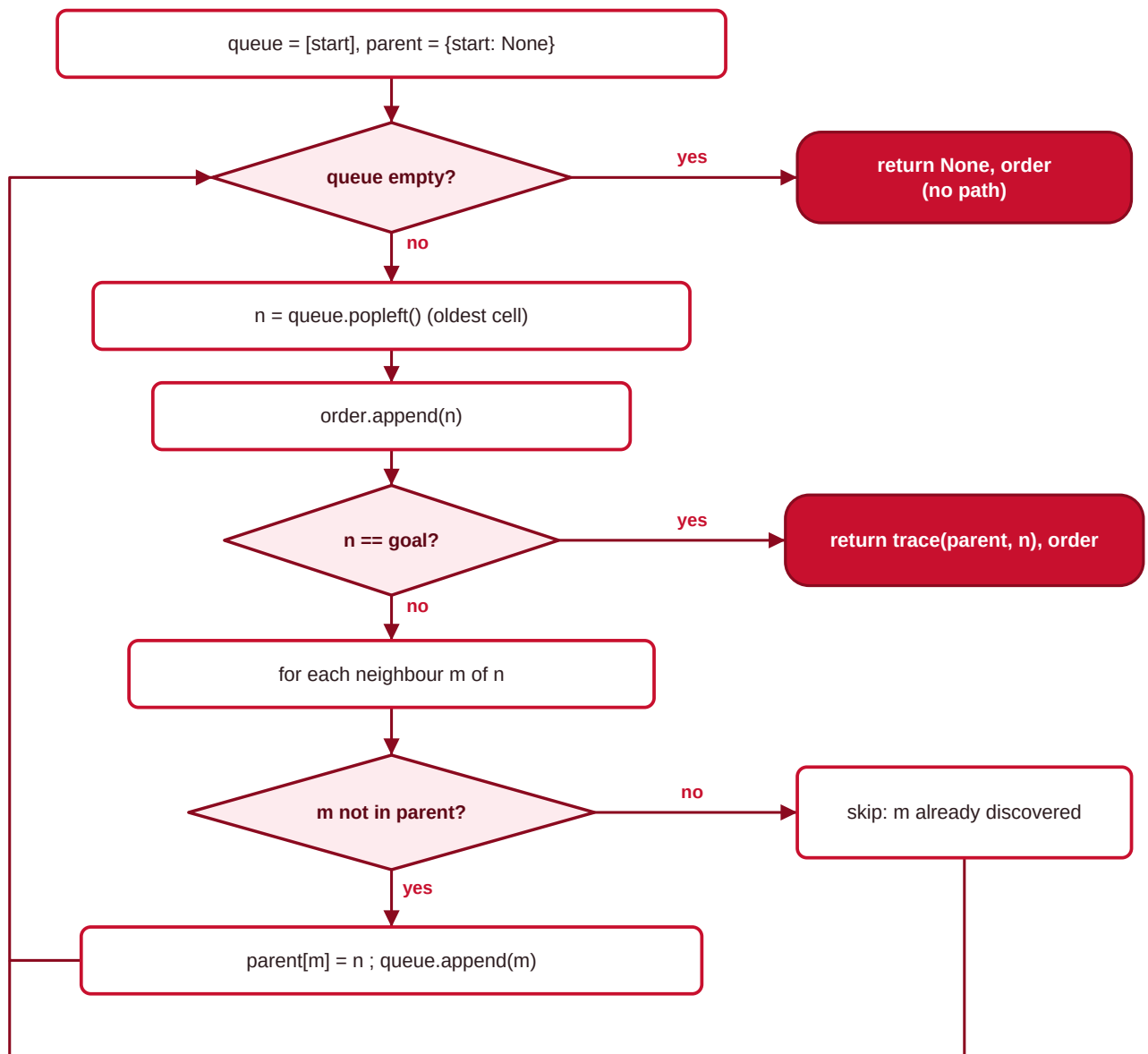
Return values: every algorithm returns two things. **path** is the list of cells from start to goal (or **None** if there is no route). **order** is the list of cells in the order they were expanded. **len(order)** is the **nodes explored** and **len(path) - 1** is the **path cost**.

3. The four algorithms

3.1 BFS: Breadth-First Search

Uses a **queue** (first in, first out). It explores the grid level by level, so the first time it reaches the goal is by the fewest possible steps. It is always optimal here because every move costs 1.

Data structure Queue (deque)	Picks next Oldest cell	Optimal? Yes
--	----------------------------------	------------------------

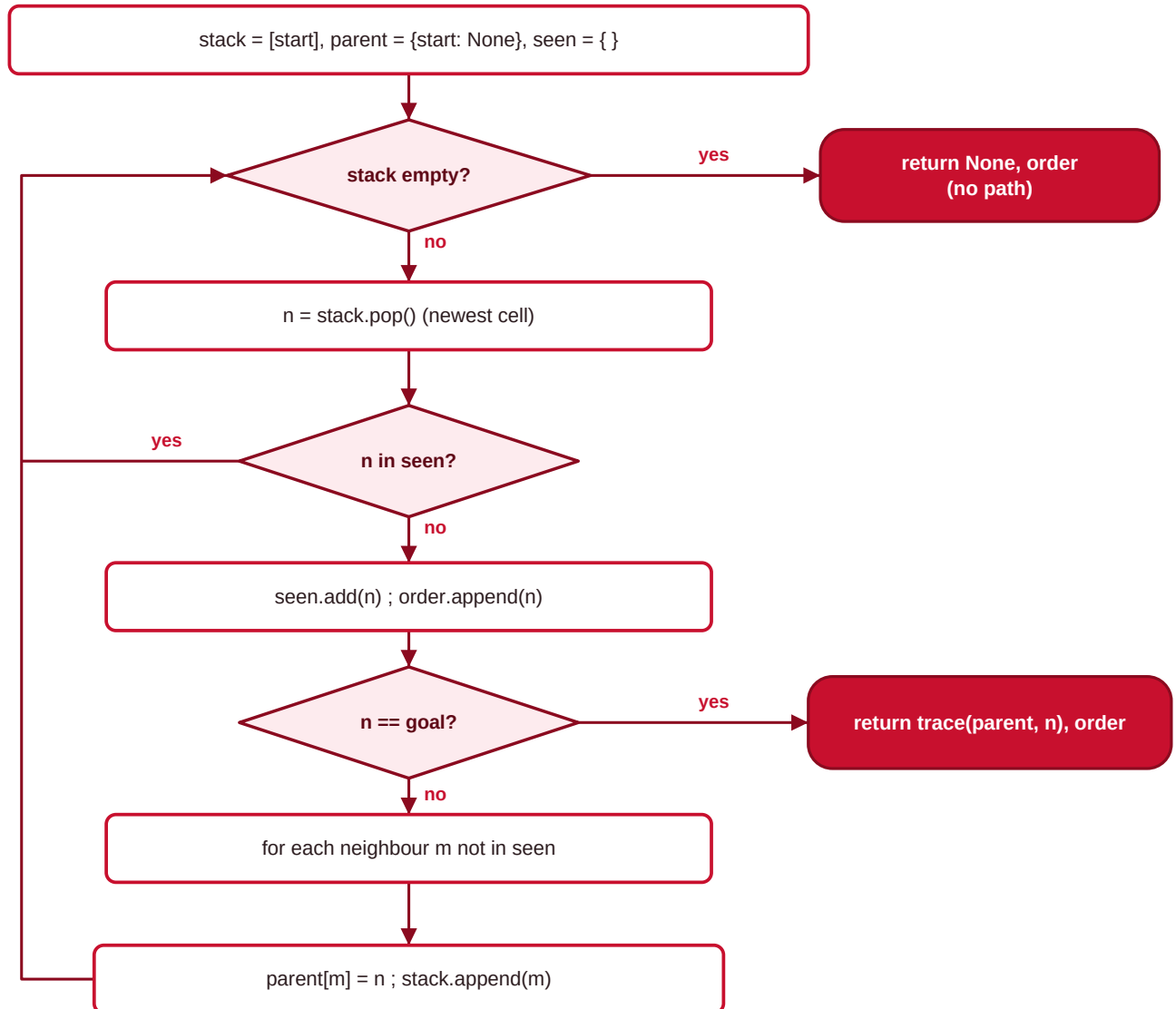


Key line: `if m not in parent` marks a cell as discovered the moment it is queued, so no cell enters the queue twice.

3.2 DFS: Depth-First Search

Uses a **stack** (last in, first out). It goes as deep as possible along one branch before backing up. It is simple and quick, but the path it finds is usually long and not optimal.

Data structure Stack (list)	Picks next Newest cell	Optimal? No
---------------------------------------	----------------------------------	-----------------------

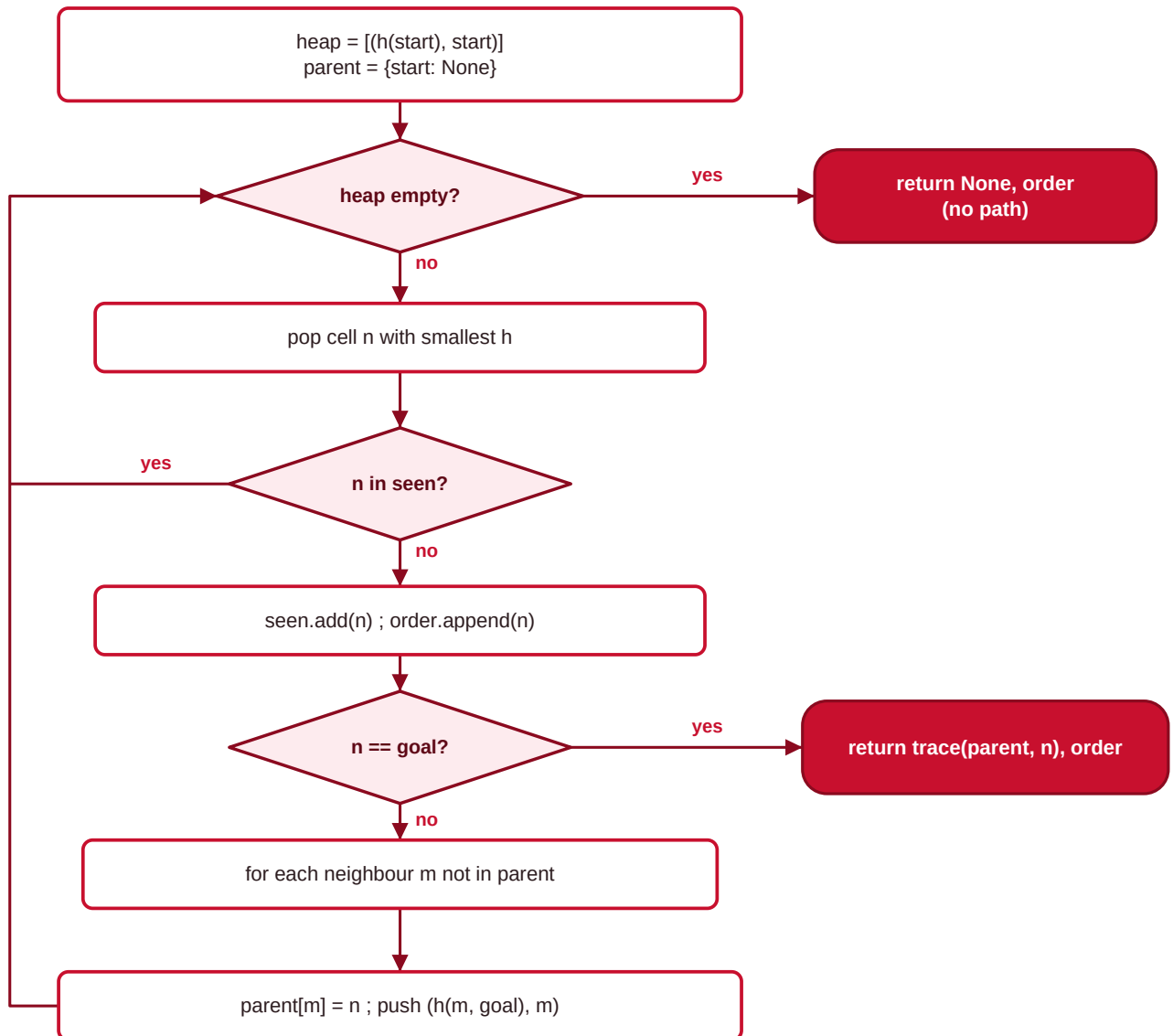


Key difference from BFS: `pop()` takes the newest item, so the agent keeps diving deeper. A cell can sit in the stack more than once, so the **seen** check after the pop skips repeats.

3.3 Greedy Best-First Search

Uses a **priority queue (min-heap)** ordered only by **$h(n, \text{goal})$** , the estimated distance left. It always expands the cell that looks closest to the goal. It is very fast, but it ignores the cost already spent, so the path is not guaranteed to be shortest.

Data structure Min-heap	Priority $f(n) = h(n)$	Optimal? No
-----------------------------------	---	-----------------------

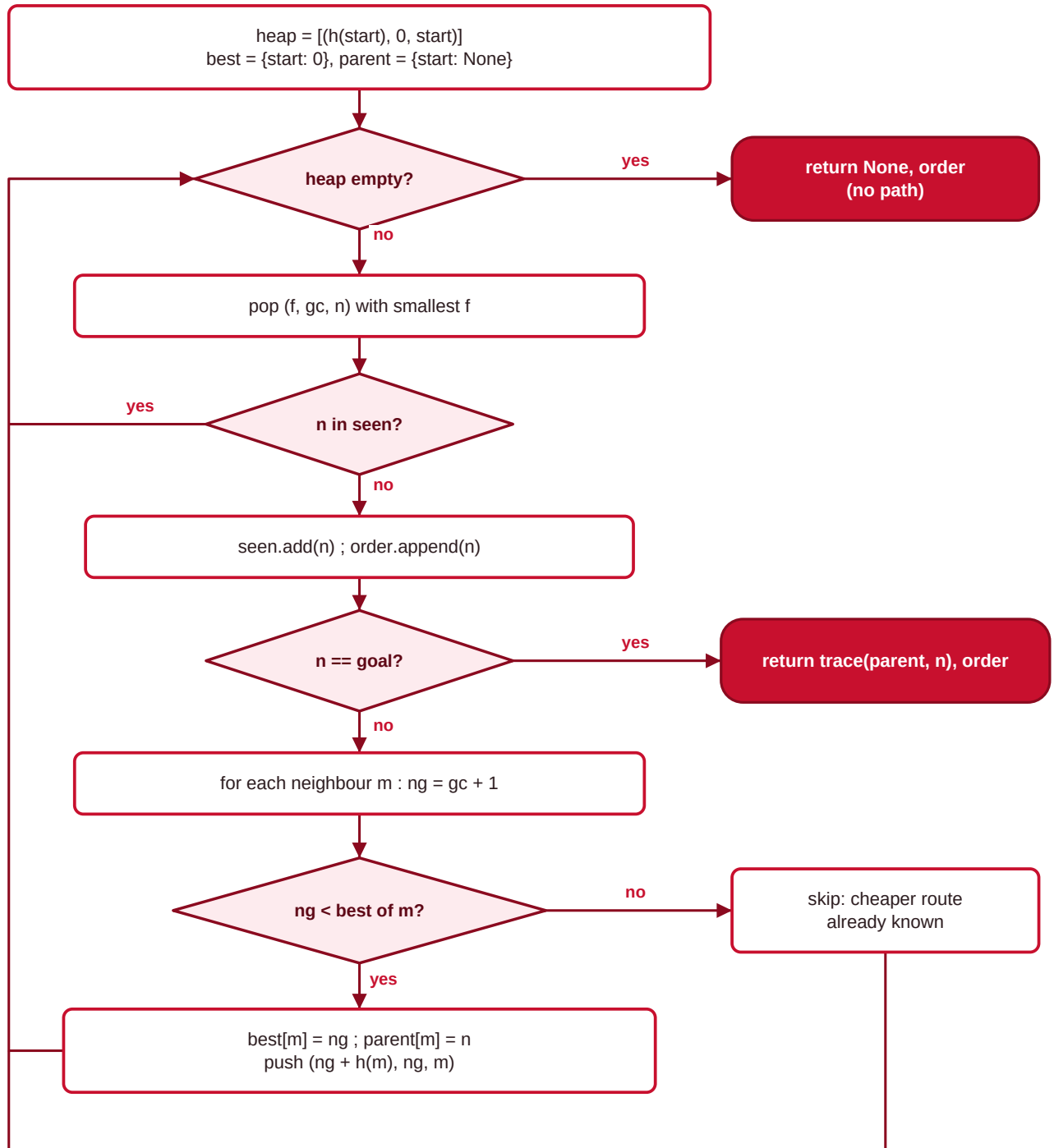


Key idea: the heap only looks **forward** (estimated distance to goal). It never checks how long the path behind it already is.

3.4 A* Search

Also a min-heap, but ordered by $f(n) = g(n) + h(n)$: the steps already taken plus the estimated steps left. Because Manhattan distance never overestimates, A* returns an **optimal** path while exploring far fewer cells than BFS.

Data structure Min-heap + best dict	Priority $f(n) = g(n) + h(n)$	Optimal? Yes
---	--	------------------------



Key line: `if ng < best.get(m, 1e9)` only accepts a route to m if it is cheaper than every route found before.

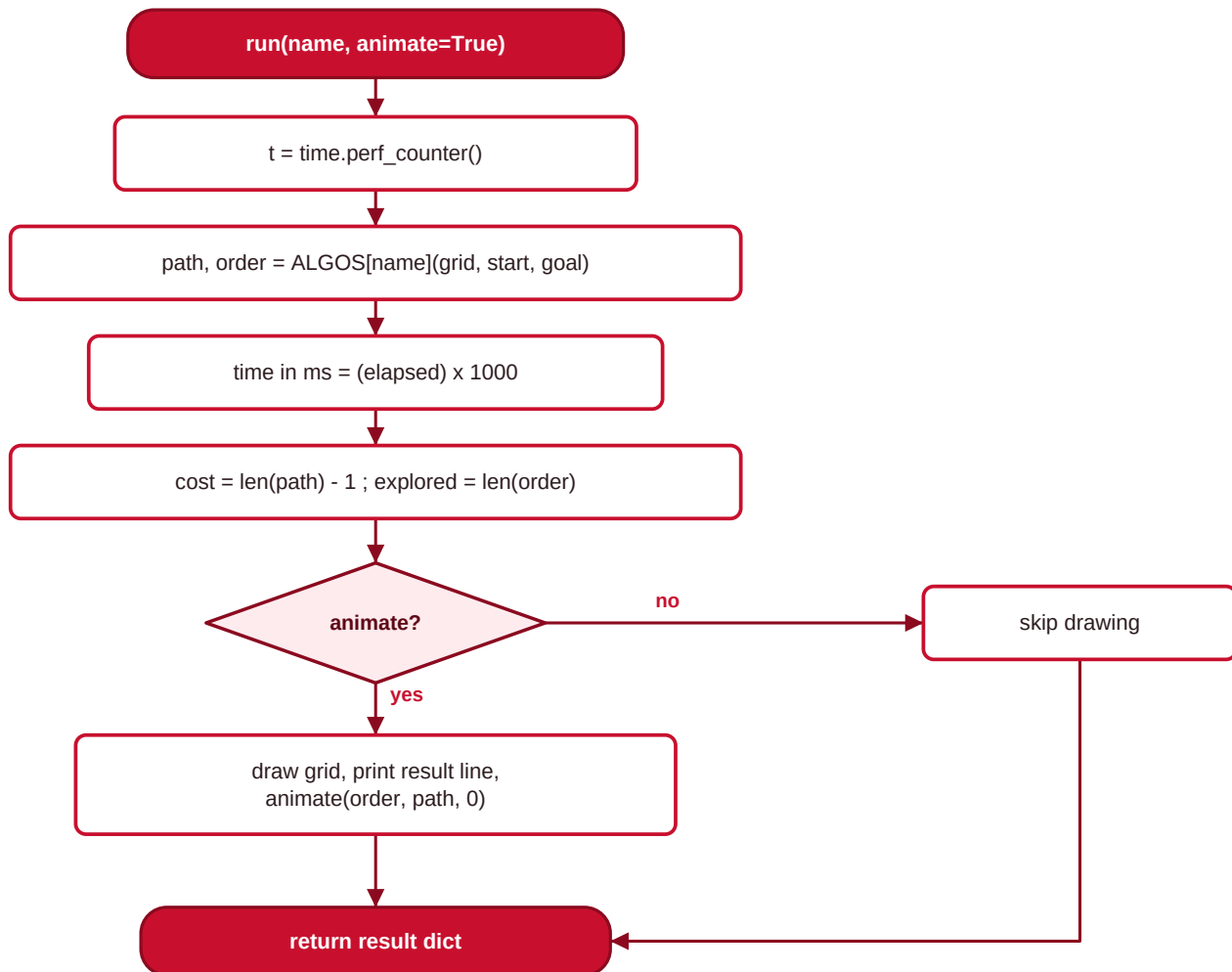
3.5 Side by side

Algorithm	Data structure	Priority	Optimal?	Speed / nodes explored
BFS	Queue (deque)	Order of discovery	Yes	Explores many nodes
DFS	Stack (list)	Newest first	No	Fast, but long path
Greedy	Min-heap	$h(n)$	No	Very few nodes, path may be longer
A*	Min-heap + best-cost dict	$g(n) + h(n)$	Yes	Optimal with few nodes

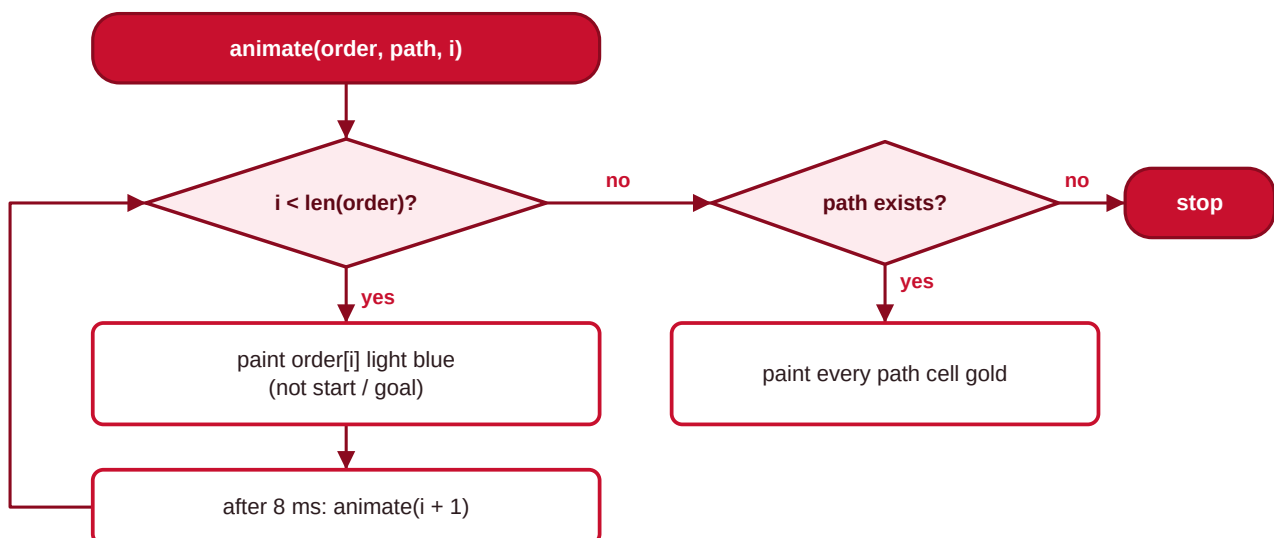
4. GUI and control functions

Function	Role
<code>App.__init__</code>	Sets start and goal, builds the canvas, the 4 "Run" buttons, "Compare All", "New Grid", the output text box, and prints the PEAS text.
<code>new_grid()</code>	Makes a 15 x 20 grid where each cell is a wall with 25% chance. Start and goal are forced free. It loops until BFS proves a path exists, so every grid is solvable.
<code>draw()</code>	Clears the canvas and paints every cell black (wall) or white (free), then the start green and the goal red.
<code>paint(p, color)</code>	Fills one cell with a colour. Used for explored cells and for the final path.
<code>run(name, animate)</code>	Calls the chosen algorithm, times it with <code>perf_counter</code> , and builds a result (cost, explored, time). If animating, it redraws, prints the result line and starts the animation.
<code>animate(order, path, i)</code>	Paints one explored cell light blue every 8 ms using <code>root.after</code> . When all are shown, it paints the final path gold.
<code>compare()</code>	Runs all four algorithms on the same grid with no animation, prints a table, and picks the best one.

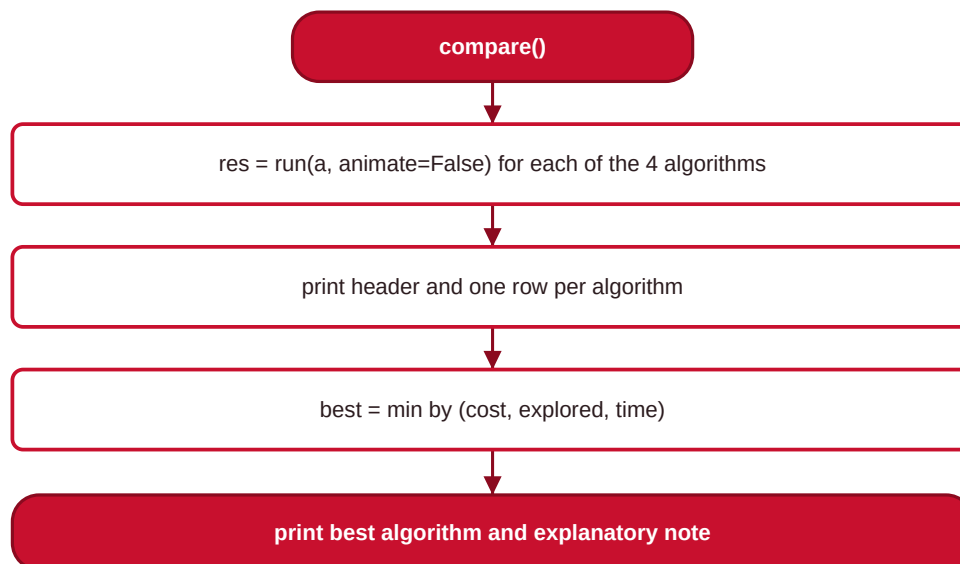
run() in detail



animate() in detail

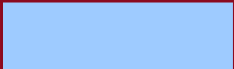


compare() in detail



Tie-break order: (cost, explored, time) means shortest path first, then fewest nodes explored, then fastest time. That is why A* usually wins.

5. Colour legend

					
White free cell	Black wall	Green start	Red goal	Light blue explored	Gold final path

6. PEAS of the agent

Performance	Path cost, nodes explored, execution time, reaching the goal
Environment	2D grid with walls and obstacles, fixed start and goal cells (fully observable, static, deterministic, discrete)
Actuators	Move up, down, left, right
Sensors	Current position, obstacle detection, goal location

7. Expected result

On a typical grid, BFS and A* give the **same shortest path cost**. A* explores far fewer cells than BFS. Greedy explores the fewest cells, but its path can be longer. DFS often finds a long, winding path. So the program reports **A* as the best choice** for this environment.